Unidad 1: Preparación del entorno de desarrollo y servidores de aplicaciones web

GitHub Pages

1. Sitios estáticos y sitios dinámicos

Un **sitio estático** es un conjunto de archivos (HTML, CSS, JavaScript, imágenes, fuentes, PDF…) que el servidor entrega exactamente tal cual están guardados, sin ejecutar ningún código propio para construir la respuesta. Si 1000 usuarios piden la misma página, los 1000 reciben el mismo archivo.

Un **sitio dinámico** genera la página en el momento de la petición: el servidor ejecuta código (PHP, Python, Ruby, Node.js, Java…), normalmente consulta una base de datos y devuelve un HTML distinto según el usuario, la hora, los datos guardados, etc.

Ejemplos típicos

- **Estáticos**: portfolio personal, documentación técnica, blog, landing page de un producto, página de un evento, currículum, apuntes como estos.
- **Dinámicos**: tienda online, red social, banca en línea, WordPress con base de datos, panel de administración, cualquier sitio con login y datos por usuario.

Diferencias

Aspecto	Sitio estático	Sitio dinámico
Contenido servido	Archivos ya creados	HTML generado en cada petición
Lenguaje en el servidor	Ninguno (solo servir ficheros)	PHP, Python, Node.js, Java, Ruby…
Base de datos	No	Normalmente sí (MySQL, PostgreSQL…)
Velocidad	Muy alta (sin procesamiento)	Depende del código y de la base de datos
Seguridad	Superficie de ataque mínima	Mayor: inyección SQL, fallos del backend, sesiones…
Escalabilidad	Trivial: se copia en una CDN	Requiere más recursos y planificación
Coste de alojamiento	Muy bajo o gratuito	Servidor con runtime y base de datos
Mantenimiento	Actualizar ficheros	Actualizar SO, runtime, CMS, dependencias, BD
Contenido por usuario	No, salvo con JavaScript + API externa	Sí, de forma nativa
Ejemplo de hosting	GitHub Pages, Netlify, S3	Apache + PHP, Node.js, Tomcat

Por qué son importantes

1. **Rendimiento**: no hay código que ejecutar ni base de datos que consultar, así que la respuesta es casi inmediata y se cachea fácilmente.
2. **Seguridad**: si no hay backend ni base de datos, no hay nada que inyectar ni que explotar en el servidor.
3. **Coste y simplicidad**: no hace falta administrar un servidor, solo alojar ficheros.
4. **Versionado**: todo el sitio son ficheros de texto, por lo que se guarda en Git con historial completo y se despliega con un simple `git push`.

Idea clave: “estático” no significa “sin interactividad”. Una página estática puede llevar JavaScript que consuma APIs externas, formularios enviados a servicios de terceros o comentarios de un servicio externo. Lo que no tiene es código de servidor propio.

2. El concepto de renderizado

Renderizar es transformar unos ingredientes (contenido, plantillas, datos) en el HTML final que se muestra al usuario. La palabra se usa en dos niveles que conviene no confundir:

- **Renderizado del contenido**: quién produce el HTML y en qué momento (un generador, el servidor o el JavaScript).
- **Renderizado del navegador**: el navegador interpreta HTML, CSS y JavaScript y lo dibuja en pantalla. Ocurre siempre, en cualquier tipo de sitio.

La diferencia real entre un sitio estático y uno dinámico está en **cuándo se genera el HTML**:

Momento	Paso 1	Paso 2	Paso 3
Compilación (una vez, al publicar)	El generador crea el HTML (build)	El servidor entrega el archivo	El navegador lo muestra
Servidor (en cada petición)	Código y base de datos generan el HTML	El servidor envía la respuesta	El navegador lo muestra
Cliente (en el navegador)	El servidor envía JavaScript y datos	El navegador ejecuta el JavaScript	El HTML se crea en el navegador

En GitHub Pages el HTML se genera en la compilación (primera fila); el servidor solo entrega ficheros ya listos. La tercera fila es JavaScript que el navegador ejecuta sobre esos ficheros, y también es compatible.

Modelos de renderizado

Modelo	Cuándo se genera el HTML	Ejemplos	¿Funciona en GitHub Pages?
Sitio estático puro	Nunca: se escribe a mano	HTML + CSS	Sí
Compilación (SSG, *Static Site Generation*)	Una vez, al publicar el sitio	Jekyll, Hugo, Eleventy	Sí, es su caso de uso principal
Servidor (SSR, *Server-Side Rendering*)	En cada petición	PHP, Django, Express, WordPress	No: no ejecuta código de servidor
Cliente (CSR, *Client-Side Rendering*)	En el navegador, tras descargar el JavaScript	React, Vue, Angular (SPA)	Sí, si el resultado se publica como ficheros estáticos

Ejemplo: de Markdown a HTML

Cuando subes un `index.md` a GitHub Pages, Jekyll lo convierte a HTML durante la compilación. Ese HTML es lo que recibe el visitante, y por eso el sitio sigue siendo estático.

```
# Hola
Esto es un **ejemplo**.
```

Se convierte en:

```
<h1>Hola</h1>
<p>Esto es un <strong>ejemplo</strong>.</p>
```

3. Generadores de sitios estáticos

Un **generador de sitios estáticos** (*Static Site Generator*, SSG) es una herramienta que toma contenido (normalmente Markdown), plantillas y configuración, y produce como resultado una carpeta de HTML, CSS y JavaScript listos para publicar. Es el paso de "compilación" o *build* de la sección anterior.

Por qué se usan

Escribir 50 páginas HTML a mano repite el mismo menú, cabecera y pie en todas. Con un generador:

- Escribes el contenido una vez en **Markdown**.
- Defines una **plantilla** (*layout*) común que se aplica a todas las páginas.
- Cambias el menú en un solo sitio y se actualiza todo el sitio.
- Obtienes ficheros estáticos, con las ventajas de rendimiento y seguridad ya vistas.

Flujo de trabajo habitual

1. Escribes contenido en Markdown + plantillas + configuración.
2. Ejecutas el generador (comando de *build*), que crea la carpeta de salida (por ejemplo `_site` o `public`).
3. Publicas esa carpeta en un servidor de ficheros estáticos (GitHub Pages, Netlify, S3, Apache, Nginx…).

Generadores más conocidos

Generador	Lenguaje	Características	Relación con GitHub Pages
Jekyll	Ruby	El más veterano; plantillas Liquid; temas y *plugins* en gemas de Ruby	Integrado: GitHub Pages lo ejecuta por defecto al publicar desde una rama
Hugo	Go	Muy rápido (miles de páginas en segundos); un único binario; temas	Se compila fuera (local o con GitHub Actions) y se publica el resultado
Eleventy (11ty)	JavaScript	Flexible, múltiples lenguajes de plantilla	Con GitHub Actions
Docusaurus	JavaScript (React)	Pensado para documentación técnica	Con GitHub Actions
MkDocs	Python	Documentación en Markdown, tema Material muy usado	Con GitHub Actions (o `mkdocs gh-deploy`)
Astro / Next.js (exportación estática) / Gatsby	JavaScript	Frameworks que pueden generar salida estática	Con GitHub Actions

Otras herramientas relacionadas

- **Builders de sitios** (Wix, Squarespace, Webflow…): editores visuales que generan el sitio por ti, pero suelen alojarlo en su propia plataforma.
- **Plataformas de alojamiento estático**: Netlify, Vercel, Cloudflare Pages o GitHub Pages. Ejecutan el *build* y sirven el resultado, normalmente desde un repositorio Git.
- **Headless CMS**: gestores de contenido sin *frontend* (Strapi, Contentful…) cuyo contenido se descarga durante el *build* para generar un sitio estático.

Builders: constructores visuales frente a generadores estáticos

La palabra *builder* se usa con dos sentidos. Los **constructores visuales** (Elementor, Divi, WPBakery, Wix, Squarespace, Webflow) permiten diseñar páginas arrastrando y soltando, sin escribir código. Las **herramientas de compilación** (Webpack, Vite, el comando `hugo` o `jekyll build`) toman código fuente y producen los ficheros finales.

Elementor es un constructor visual que funciona como *plugin* de WordPress. WordPress guarda el diseño en una base de datos y genera el HTML en cada petición, así que el resultado es un sitio **dinámico**.

Grupo	Ejemplos	Cómo se trabaja	Resultado	¿Funciona en GitHub Pages?
Generadores estáticos	Jekyll, Hugo, MkDocs	Ficheros de texto (Markdown y plantillas) y un comando de *build*	Sitio estático	Sí
Constructores visuales sobre un CMS	Elementor, Divi, WPBakery (WordPress)	Arrastrar y soltar dentro del CMS	Sitio dinámico (PHP y base de datos)	No
Plataformas cerradas todo en uno	Wix, Squarespace, Webflow	Editor visual en línea con alojamiento incluido	Sitio alojado en la propia plataforma	No (algunas permiten exportar el código, según el plan)
Herramientas de compilación	Webpack, Vite, Parcel	Empaquetan el JavaScript y CSS de una aplicación	Ficheros estáticos	Sí, publicando el resultado

Elementor no puede alojarse en GitHub Pages porque depende de WordPress, que necesita PHP y una base de datos. Si quieres un sitio visual y estático, hay que exportarlo a HTML con herramientas de terceros (con las limitaciones que eso tenga) o usar un generador estático.

¿Y cuál elijo?

- Quieres **algo sencillo** en GitHub Pages sin instalar nada: Markdown + Jekyll integrado.
- Quieres **máxima velocidad de compilación** y sitios grandes: Hugo.
- Quieres **documentación técnica**: MkDocs o Docusaurus.
- Solo necesitas **una página** o una landing: HTML y CSS a mano, sin generador.

4. Qué es GitHub Pages y qué rol juega

GitHub Pages es un servicio de alojamiento de sitios estáticos de GitHub. Toma archivos HTML, CSS y JavaScript directamente de un repositorio, opcionalmente los pasa por un proceso de compilación (por defecto, Jekyll) y publica el resultado como un sitio web.

Puedes hospedar el sitio en el dominio `github.io` de GitHub o en un **dominio personalizado** propio.

El rol de GitHub Pages en el esquema anterior

GitHub Pages cubre **dos papeles a la vez**:

1. **Servidor web estático**: recibe las peticiones de los navegadores y devuelve los ficheros del repositorio.
2. **Plataforma de compilación** (opcional): ejecuta Jekyll, o un flujo de GitHub Actions que tú definas, para generar el HTML a partir de tu Markdown.

El **repositorio de Git** actúa como "panel de control": editas ficheros, haces `git push` y el sitio se actualiza solo. Es un despliegue continuo sin configurar un servidor.

Ventajas

- Gratuito para repositorios públicos.
- Despliegue automático con cada `git push`.
- Historial completo de cambios en Git (puedes volver a cualquier versión anterior).
- HTTPS incluido y dominios personalizados.
- Sin servidor que administrar ni parchear.

Limitaciones

- **No soporta lenguajes del lado del servidor** como PHP, Ruby o Python: no puedes ejecutar tu propio backend.
- **No hay base de datos** propia. Si necesitas datos dinámicos, tendrás que consumir una API externa desde JavaScript.
- **Sitios y ancho de banda con límites de uso** (GitHub publica sus límites en la documentación oficial y pueden cambiar).
- Está pensado para **sitios personales, de proyecto, documentación o portfolio**, no para tiendas online ni servicios comerciales.
- Un sitio publicado desde un repositorio público es **visible para todo el mundo**, aunque la política de repositorios privados depende del plan contratado.

> **Para clase**: GitHub Pages nos sirve para practicar el ciclo completo de publicación web (repositorio, *commit*, *push*, despliegue, comprobación) sin montar un servidor. Lo que aprendemos aquí se traslada directamente a otros servicios de alojamiento estático.

5. Tipos de sitios de GitHub Pages

Existen tres tipos de sitios, que se distinguen por el **nombre del repositorio** y por la **URL** resultante.

Tipo	Se conecta a	Nombre del repositorio	URL del sitio	¿Cuántos?
Usuario	Una cuenta personal	`username.github.io`	`https://username.github.io`	Uno por cuenta
Organización	Una organización	`organization.github.io`	`https://organization.github.io`	Uno por organización
Proyecto	Un repositorio concreto	Cualquiera (p. ej. `mi-proyecto`)	`https://username.github.io/repositorio`	Ilimitados

Detalles importantes

- En los sitios de **usuario y organización**, el repositorio debe llamarse exactamente `username.github.io` (con tu nombre de usuario real, en minúsculas).
- En los sitios de **proyecto**, los ficheros fuente del sitio viven en el **mismo repositorio** que el proyecto, y la URL incluye el nombre del repositorio como ruta.
- Un mismo usuario puede tener **un** sitio de usuario y **muchos** sitios de proyecto a la vez.

Ejemplo

Si tu usuario es `maria`:

Repositorio	URL resultante
`maria.github.io`	`https://maria.github.io/`
`iaw_202627`	`https://maria.github.io/iaw_202627/`
`portfolio`	`https://maria.github.io/portfolio/`

6. Fuentes de publicación

La **fuente de publicación** indica a GitHub Pages **de dónde** tiene que coger los ficheros del sitio. Hay tres opciones:

Fuente	Dónde están los ficheros	Cuándo usarla
Carpeta `/docs` de la rama `main`	Carpeta `docs/` en la raíz del repositorio	Cuando el repositorio mezcla código y sitio web, o quieres una sola rama
Raíz `/` de la rama `gh-pages`	Raíz de una rama dedicada llamada `gh-pages`	Cuando prefieres separar el sitio del resto del código en una rama propia
GitHub Actions	Lo decide un flujo de trabajo (*workflow*)	Cuando usas un generador distinto de Jekyll (Hugo, MkDocs…) o un *build* personalizado

Publicar desde una rama

Si no necesitas controlar el proceso de creación del sitio, lo más sencillo es **publicar desde una rama**. Eliges rama y carpeta, y cada vez que hagas `push` a esa rama, los cambios de esa carpeta se publican en el sitio.

- **Opción 1: `/docs` de `main`**. Todo el trabajo está en la misma rama; la carpeta `docs/` contiene solo el sitio.
- **Opción 2: raíz de `gh-pages`**. La rama `gh-pages` contiene únicamente los ficheros del sitio. Es la opción que usamos en clase con el repositorio `iaw_202627`.

Crear la rama `gh-pages` desde la terminal:

```
git checkout -b gh-pages      # crea la rama y te mueve a ella
git push -u origin gh-pages   # la sube a GitHub y la enlaza con el remoto
```

Con la sintaxis moderna sería `git switch -c gh-pages`.

> **Ojo**: al trabajar en `gh-pages`, tus *commits* y `push` van a esa rama. Comprueba siempre con `git status` en qué rama estás antes de subir cambios, tal como vimos con `HEAD` y las ramas.

Publicar con GitHub Actions

Si usas Hugo, MkDocs u otro generador, el sitio no se puede compilar directamente desde la rama. En ese caso defines un **flujo de GitHub Actions** (un archivo `.yml` en `.github/workflows/`) que compila el sitio y lo publica. GitHub ofrece plantillas de flujos de trabajo para los generadores más habituales.

El archivo de entrada

GitHub Pages busca como página principal un archivo llamado `index.html`, `index.md` o `README.md`, y debe estar en el **nivel superior de la carpeta de origen**:

- Fuente `/docs` de `main` → `docs/index.md`
- Fuente raíz de `gh-pages` → `index.md` en la raíz de la rama `gh-pages`

7. Crear un sitio paso a paso

Antes de crear el sitio necesitas un repositorio en GitHub que lo contenga.

Paso 1: crear el repositorio

1. En la esquina superior derecha de cualquier página de GitHub, pulsa **New** (Nuevo repositorio).
2. Elige el propietario (*Owner*) y escribe el nombre:
   - Sitio de **usuario u organización**: `username.github.io` u `organization.github.io`.
   - Sitio de **proyecto**: el nombre que quieras.
3. Marca el repositorio como **Public** (necesario en cuentas gratuitas).
4. Marca **Add a README file** para que se cree el primer *commit*.
5. Pulsa **Create repository**.

Paso 2: clonar el repositorio

```
git clone https://github.com/username/username.github.io.git
cd username.github.io
```

En VS Code puedes hacerlo desde la paleta de comandos con **Git: Clone**, como vimos.

Paso 3: decidir la fuente de publicación

Elige una de las opciones de la sección 6. Si vas a usar la rama `gh-pages`, créala antes:

```
git checkout -b gh-pages
```

```
git push -u origin gh-pages
```

Paso 4: crear el archivo de entrada

Crea `index.md` (o `index.html`) en el nivel superior de la carpeta de origen. Un ejemplo mínimo:

```
# Mi sitio en GitHub Pages

Bienvenido a mi página personal.

## Sobre mí

- Estudiante de 2º ASIR

- Módulo: Implantación de Aplicaciones Web
```

Paso 5: subir los cambios

```
git add index.md
```

```
git commit -m "Añado página principal"

git push
```

O desde VS Code: **Source Control** → + para añadir al área de preparación → mensaje → **Commit** → **Sync Changes** o **Push**.

Paso 6: activar GitHub Pages

1. En el repositorio, entra en **Settings** (Configuración).
2. En la barra lateral, sección **Code and automation**, pulsa **Pages**.
3. En **Build and deployment → Source**, elige **Deploy from a branch**.
4. En **Branch**, selecciona la rama (`main` o `gh-pages`) y la carpeta (`/docs` o `/ (root)`).
5. Pulsa **Save**.

Paso 7: ver el sitio publicado

En la misma pantalla de **Pages** aparecerá un mensaje con la URL del sitio y un botón **Visit site**.

> **Espera unos minutos**: la publicación puede tardar hasta 10 minutos tras el `push`. Si pasada una hora no ves los cambios, probablemente hay un error en los documentos (ver sección 11).

Qué ocurre por detrás

Cada `push` a la rama de publicación dispara un flujo de **GitHub Actions** que compila el sitio (con Jekyll, si no has configurado otra cosa) y lo despliega. Puedes ver su progreso y sus errores en la pestaña **Actions** del repositorio.

8. Jekyll en GitHub Pages, temas y `.nojekyll`

Cuando publicas desde una rama, GitHub Pages ejecuta **Jekyll** por defecto. Por eso un simple `index.md` termina convertido en una página HTML.

Qué hace Jekyll por ti

- Convierte los ficheros Markdown (`.md`) en HTML.
- Aplica una plantilla (*layout*) y, si lo configuras, un **tema**.
- Procesa los archivos que empiezan con un bloque de configuración llamado *front matter*.
- Ignora los ficheros y carpetas que empiezan por `_` o `.` (salvo excepciones configuradas).

Front matter

Es un bloque entre dos líneas `---` al principio del archivo, con metadatos de la página en formato YAML:

```
---
layout: default
title: Mi primera página
---

# Hola

Contenido de la página.
```

Añadir un tema

Para cambiar el aspecto sin escribir CSS, crea un archivo `_config.yml` en la raíz de la fuente de publicación:

```
theme: jekyll-theme-minimal
title: Mi sitio
```

GitHub Pages admite un conjunto de temas oficiales (por ejemplo `minimal`, `cayman`, `slate`, `midnight`…). También puedes elegir un tema desde **Settings → Pages** en la interfaz de GitHub.

Desactivar Jekyll: el archivo `.nojekyll`

Si tu sitio ya está compilado (por ejemplo, la carpeta de salida de Hugo, o HTML escrito a mano) y no quieres que Jekyll lo procese, crea un **archivo vacío llamado `.nojekyll`** en la raíz de la fuente de publicación:

```
touch .nojekyll
git add .nojekyll
git commit -m "Desactivo Jekyll"
git push
```

En Windows PowerShell puedes crearlo con `New-Item .nojekyll -ItemType File`.

Resumen: ¿cuándo cada opción?

Situación	Qué hacer
Sitio en Markdown y HTML sencillo	Nada: Jekyll se ejecuta solo
Quiero un tema listo	`theme:` en `_config.yml`
Sitio ya compilado con otra herramienta	Añadir `.nojekyll`
Uso Hugo, MkDocs u otro generador	Flujo de GitHub Actions

9. Añadir páginas y estructura del sitio

Cada archivo que creas en la fuente de publicación se convierte en una página del sitio, con la **misma estructura de directorios**. Los archivos `.md` se publican con extensión `.html`.

Archivo en el repositorio	URL en el sitio (proyecto `maria.github.io/iaw_202627`)
`index.md`	`https://maria.github.io/iaw_202627/`
`about.md`	`https://maria.github.io/iaw_202627/about.html`
`about/contact-us.md`	`https://maria.github.io/iaw_202627/about/contact-us.html`
`img/foto.png`	`https://maria.github.io/iaw_202627/img/foto.png`

Estructura de ejemplo

```
mi-sitio/
├── index.md
├── about.md
├── _config.yml
├── css/
│   └── estilos.css
├── img/
│   └── foto.png
└── practicas/
```

```
├── practica1.md
└── practica2.md
```

Enlazar entre páginas

Usa rutas relativas para que el sitio funcione tanto en local como en GitHub Pages:

```
[Sobre mí](about.md)
[Práctica 1](practicas/practica1.md)
![Mi foto](img/foto.png)
```

> **Ojo con las mayúsculas**: los servidores de GitHub distinguen entre mayúsculas y minúsculas. `Foto.png` y `foto.png` son ficheros distintos, y un enlace mal escrito funciona en Windows pero falla al publicar.

10. Dominio personalizado y HTTPS

Por defecto tu sitio vive en `username.github.io`. Puedes servirlo desde un dominio propio (por ejemplo `www.midominio.es`) configurando tres piezas: el **DNS**, GitHub y el fichero `CNAME`.

Tipos de dominio y registros DNS

Tipo de dominio	Ejemplo	Registro DNS que se configura
Subdominio	`www.midominio.es`	CNAME apuntando a `username.github.io`
Dominio raíz (*apex*)	`midominio.es`	Registros A (y AAAA) hacia las IP que indica la documentación de GitHub

Las direcciones IP concretas cambian con el tiempo; consúltalas siempre en la documentación oficial de GitHub Pages antes de configurar el DNS.

Pasos

1. En el panel de tu proveedor de dominio, crea el registro DNS correspondiente.
2. En el repositorio, ve a **Settings → Pages → Custom domain**, escribe el dominio y pulsa **Save**.
3. GitHub crea (o actualiza) un archivo `CNAME` en la fuente de publicación con el dominio dentro.
4. Espera a que se propague el DNS (puede tardar desde minutos hasta horas).
5. Activa **Enforce HTTPS** cuando esté disponible.

HTTPS

GitHub Pages emite y renueva automáticamente un certificado para tu sitio. En `github.io` el HTTPS viene activado. En un dominio propio, la casilla **Enforce HTTPS** se habilita cuando el certificado está listo. Así el tráfico entre el navegador y el sitio va cifrado.

> **Comprobar el DNS desde clase**: `nslookup www.midominio.es` o `dig www.midominio.es` te muestran a dónde apunta el dominio.

11. Problemas frecuentes y cómo depurarlos

Síntoma	Causa probable	Solución
Error 404 al abrir el sitio	No existe el archivo de entrada, o está fuera de la carpeta de origen	Crea `index.md` o `index.html` en la raíz de la fuente (`/docs` o raíz de `gh-pages`)
Subí cambios y no se ven	Publicación en curso, o el `push` fue a otra	Espera unos minutos; comprueba la rama con

Síntoma	Causa probable	Solución
	rama	`git status` y la pestaña **Actions**
El sitio no se actualiza tras una hora	Error en el *build* de Jekyll	Revisa el fallo en la pestaña **Actions**
Faltan estilos o imágenes	Rutas absolutas o mayúsculas/minúsculas distintas	Usa rutas relativas y respeta exactamente el nombre de los ficheros
Un fichero o carpeta no aparece	Su nombre empieza por `_` o `.` y Jekyll lo ignora	Cámbiale el nombre o añade `.nojekyll` si no usas Jekyll
La lista anidada o la tabla se ven mal	Sintaxis de Markdown incorrecta (indentación, línea en blanco)	Revisa con la vista previa de VS Code antes de hacer `push`
No puedo elegir *Deploy from a branch*	Repositorio privado en un plan que no lo permite	Hazlo público o revisa tu plan de GitHub
El dominio propio da error de certificado	El DNS aún no se ha propagado	Espera y vuelve a intentar activar **Enforce HTTPS**

Cómo depurar de forma ordenada

1. Comprueba que estás en la **rama correcta** y que hiciste *commit* y *push*.
2. En GitHub, abre la pestaña **Actions** y mira si el último flujo terminó en verde o en rojo.
3. Si hay error, abre el registro del flujo y busca la línea que lo describe.
4. Ve a **Settings → Pages** y verifica rama y carpeta de origen.
5. Abre la URL en una ventana de incógnito para descartar la caché del navegador.